

Балтийский государственный технический университет
«ВОЕНМЕХ» им. Д. Ф. Устинова

Кафедра О7 «Информационные системы и программная инженерия»

Практическая работа №1
по дисциплине «Структуры и организация данных»
на тему «Линейные структуры данных»

вариант 12

Выполнил:
Студент Праудин Д. С.
Группа О717Б

Преподаватель:
Палехова О. А.

Санкт-Петербург
2022 г.

Задача 1.

Уровень сложности – **средний**. Данные хранятся в бинарном файле записей, а для обработки считываются в односвязный линейный список (если файл не существует, то создается пустой список). При выходе из программы обработанные данные сохраняются в том же файле. Имя файла с данными должно передаваться программе при ее запуске (через параметры функции `main()`). Если параметры пользователем при запуске программы не заданы, имя файла задается константой. Элементами данных должны являться записи сложной структуры (дополнительные поля придумать самостоятельно). Обязательные операции для списка: добавление элемента в конец списка, просмотр списка, удаление последнего элемента списка, поиск в соответствии с индивидуальным вариантом. Вывод данных осуществлять в табличном виде с графлением визуально подходящими символами.

Поля данных: наименование товара, страна-импортер и объем поставляемой партии в штуках. Вывести страны, в которые экспортируется определенный товар и общий объем его экспорта.

Созданные типы данных:

`struct country` – тип данных созданный, как дополнительная структура

`country_1` – страна экспортер

`country_2` – страна импортер

`struct product` – тип данных, созданный для хранения всей информации о закупках товаре (для удобства в написании кода с помощью `typedef` заменил `struct product` на `DataType`);

`product_name` – имя товара

`struct country countries` – дополнительная структура

`amount` – объем

`struct node` – создан для хранения информации об узле (данные в узле и указатель на следующий узел)

`data` – данные в узле

`struct node * next` – указатель на следующий узел

Используемые функции:

`DataType input_product (void)` – функция заполнения полей (входные данные – ничего; выходные – объект структуры)

`list read_file (char * , list *)` – функция считывания из файла (входные данные - имя файла, указатель указателя на начало списка ; выходные –указатель на последний узел(end))

`list new_node (list *, list , DataType)` – функция добавления узла (входные данные - указатель указателя на первый узел, указатель на конец списка, поля структуры; выходные данные - указатель на последний узел(end))

`int write_file (char * , list)` – функция записи в файл (входные данные –имя файла, указатель на начало; выходные – возвращается 1, если запись произошла успешно)

`void delete_list (list)` – функция удаления всего списка (входные данные – указатель на начало; выходные – ничего)

`void show (list)` – функция вывода списка (входные данные – указатель на узел; выходные данные – ничего)

`void search (list begin)` – функция поиска по имени продукта (входные данные – узел; выходные данные – ничего)

`list delete_node (list *, list)` – функция удаления последнего узла (входные данные - указатель указателя на первый узел, указатель на конец; выходные - указатель на последний узел(end))

Текст программы:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>
#include <windows.h>
struct country // дополнительная структура
{
    char country_1[64];
    char country_2[64];
};

struct product
{
    char product_name[64];
    struct country countries;
    int amount;
};

typedef struct product DataType;

struct node //узел
{
    DataType data;
    struct node * next;
};

typedef struct node * list; //адрес первого узла
DataType input_product (void); //заполнение полей
```

```

list read_file (char * , list *); //считывание из файла
list new_node (list *, list, DataType); //добавление узла
int write_file (char * , list); // запись в файл
void delete_list (list); // удаление всего списка
void show (list); // вывод списка
void search (list begin); // поиск по имени продукта
list delete_node (list *, list); //удаление последнего узла

int main (int argc, char *argv[]) //кол.во аргументов и
{
    SetConsoleCP(1251);
    SetConsoleOutputCP (1251);
    char file[50];
    if (argc > 1) //Если в аргументы main что-то передано,
        strcpy(file, argv[1]); //то записать это в строку для названия
файла
    else
    {
        puts ("Enter the file name");
        gets(file); // Иначе, попросить пользователя осуществить ввод
имени файла с клавиатуры
    }
    char menu;
    list countrys = NULL;
    list end = NULL;
    end = read_file (file, &countrys);
    do
    {
        system ("CLS");
        puts ("1. Insert");
        puts ("2. Show");
        puts ("3. Search");
        puts ("4. Delete_end");
        puts ("5. Exit");

        menu = getchar(); getchar();
        system ("CLS");
        switch (menu)
        {
            case '1': end = new_node (&countrys, end, input_product());
break; //добавить узел
            case '2': show (countrys); break; // вывести список
            case '3': search (countrys); break; // поиск по имени
продукта
            case '4': end = delete_node (&countrys, end);break;
//удаление узла
        }
    }
    while (menu!='5');
    if (write_file (file, countrys)) // если в файл что-то записалось
        puts ("File saved");
    else
        puts ("File not saved");
    delete_list (countrys); // удаление списка
    return 0;
}

DataType input_product (void) // заполнение полей
{
    DataType product;
    puts ("product Name");
    gets (product.product_name);
    puts ("country_1");
}

```

```

        gets (product.countries.country_1);
        puts ("country_2");
        gets (product.countries.country_2);
        puts ("amount");
        scanf ("%d", &product.amount);
        getchar();
        return product;
    }

    list new_node (list * begin, list end, DataType product) // добавление
узла (указатель указателя на первый узел, указатель на конец, поля структуры )
    {
        list temp = (list) malloc (sizeof(struct node)); // выделяем память
под узел
        temp->data = product; //записываем поля структуры
        temp->next = NULL; //указатель на следующий элемент

        if(!(*begin)) // если в списке нету элементов (*begin адрес первого
узла)
        {
            *begin = temp; // адрес первого элемента
            (*begin)->next = NULL;
            end = temp;
            end->next = NULL; // т.к.добавили в конец указатель next = NULL
        }
        else // если есть элементы в списке
        {
            end->next = temp; //добавление узла в конец
            end = end->next; //меняем указатель т.к добавили элемент
        }

        return end;
    }

    list delete_node (list * begin, list end) //удаление последнего узла
(указатель указателя на первый узел, указатель на конец)
    {
        if (*begin) //если список не пуст
        {
            if ((*begin)->next) //если узел не один
            {
                list temp_1 = *begin;
                list temp_2 = temp_1->next;
                while(temp_2->next) //перебор все узлов temp_1 текущий,
temp_2 следующий
                {
                    temp_1 = temp_1->next;
                    temp_2 = temp_2->next;
                }
                free(end);
                end = temp_1; // при удалении узла указатель на конец с
temp_2 переходит на temp_1
                end->next = NULL;
            }
            else // если один узел
            {
                free(*begin);
                *begin = NULL;
                end = NULL;
            }
            puts ("Deleted");
            getchar();
        }
    }

```

```

        return end;
    }

    list read_file (char * filename, list * begin) // считываем из файла и
добавляем в список (имя файла, указатель указателя на начало)
    {
        FILE * f;
        DataType product;
        list end=NULL; // указатель на конец (список пустой)
        if ((f = fopen (filename, "rb")) == NULL)
        {
            perror ("Error open file");
            getchar();
            return end;
        }
        while (fread(&product, sizeof(product), 1, f)) // пока в файле что-то
есть то добавляем в список
            end = new_node(begin, end, product);
        fclose(f);
        return end;
    }

    void delete_list (list begin) // передаем (указатель на начало) удаление
всего списка
    {
        list temp = begin;
        while (temp) //проходимся по всем узлам
        {
            begin = temp->next;
            free (temp);
            temp = begin;
        }
    }

    int write_file (char * filename, list begin) //запись в файл (имя файла,
указатель на начало)
    {
        FILE * f;
        if ((f = fopen (filename, "wb")) == NULL) // проверка на открытие
бинарного файла
        {
            perror ("Error create file");
            getchar();
            return 0;
        }
        while (begin) // пока текущий указатель !NULL (перебор всего списка)
        {
            if (fwrite (&begin->data, sizeof(DataType), 1, f)) //если
записалось в файл, то переходим к следующему элементу
                begin = begin->next; // переставляем указатель
            else
            {
                fclose(f);
                return 0;
            }
        }
        fclose(f);
        return 1;
    }

    void print_data (struct product product) //ВЫВОД
    {
        printf ("Name : %s\n product_name: %s\n country_1 : %s\n country_2 :
%s\n amount : %.2f\n",
                product.product_name,

```

```

product.countries.country_1,product.countries.country_2, product.amount);
}

void show (list cur) //Вывод на экран списка в виде таблички
{
    int k=0;
    if (cur == NULL)
    {
        puts ("List is empty");
        getchar();
        return;
    }
    puts ("|   N |               Name               |   counry_1   |
counry_2 |   amount   | ");
    puts ("-----");
    while (cur)
    {
        printf ("|   %d | %-28s | %-14s | %-13s | %-9d |\n", ++k,
                cur->data.product_name,
                cur->data.countries.country_1,cur->data.countries.country_2, cur->data.amount);
        cur = cur->next;
    }
    puts ("-----");
    getchar();
}

void search (list cur) //поиск по имени продукта
{
    int sum=0;
    char name[50];
    DataType product;
    if (cur == NULL)
    {
        puts ("List is empty");
        getchar();
        return;
    }
    puts ("name");
    scanf ("%s",&name);
    getchar();
    while (cur)
    {
        if(strcmp(cur->data.product_name, name) == 0) //если нашлось имя
        введенное с клавиатуры
        {
            printf("%s  %s  %s  %d  \n", cur->data.product_name,cur-
            >data.countries.country_1,cur->data.countries.country_2, cur->data.amount);
            sum+=cur->data.amount; // общий объем экспорта продукции
        }

        cur = cur->next;
    }
    printf("объем экспорта: %d",sum);
    printf("\n");
    system ("pause");
}

```

Результаты работы программы:

```
1. Insert
2. Show
3. Search
4. Delete_end
5. Exit
```

Запуск программы (имя файла передается в качестве аргумента в main)

1.Выбираем пункт 1.Insert (добавление элемента в список)

Вводим поля :имя, страна экс, страна имп, объем экспорта

```
product Name
Яблоки
country_1
Польша
country_2
Россия
amount
100_
```

```
product Name
Груши
country_1
Аргентина
country_2
Россия
amount
12_
```

```
product Name
Яблоки
country_1
Китай
country_2
Россия
amount
50_
```

2.Вибраем пункт 2.Show (вывод списка в виде таблицы)

N	Name	country_1	country_2	amount
1	Яблоки	Польша	Россия	100
2	Груши	Аргентина	Россия	12
3	Яблоки	Китай	Россия	50

3.Выбираем пункт 3.Search(поиск по имени продукта и вывод общего объема)


```
name
Яблоки
Яблоки Польша Россия 100
Яблоки Китай Россия 50
объем экспорта: 150
```

4.Выбираем пункт 4.Delete_end(удаление с конца)

```
Deleted
```

N	Name	country_1	country_2	amount
1	Яблоки	Польша	Россия	100
2	Груши	Аргентина	Россия	12

5.Выбираем 5.Exit(выход из программы)

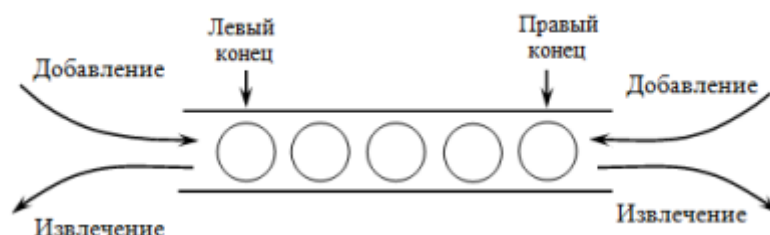
```
File saved
```

Задача 2. Структура данных –дек

Уровень сложности – **средний**. Реализовать необходимую структуру данных с помощью одной структуры хранения (векторной или связной). Реализацию оформить в виде класса. Использовать созданный тип данных для решения поставленной задачи.

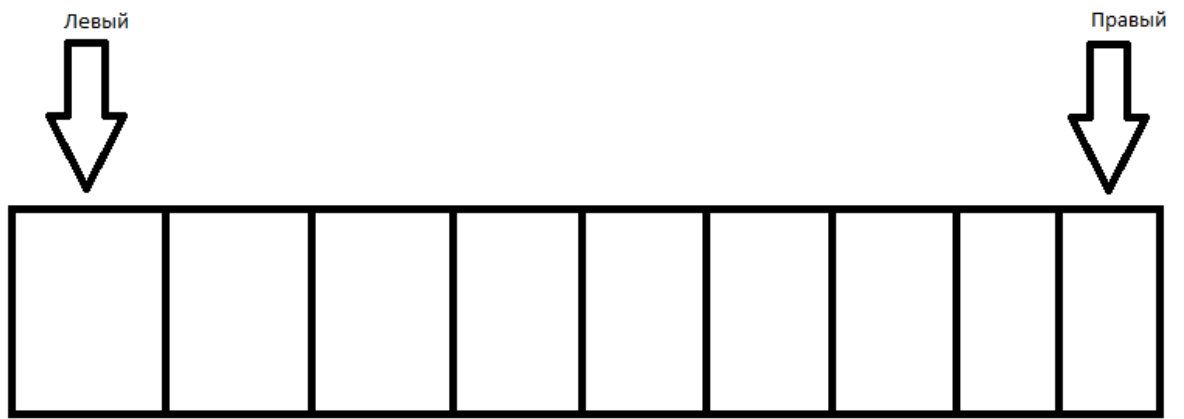
Элементами дека являются натуральные числа. Удалите из дека элементы, являющиеся простыми числами. Расположите оставшиеся элементы в порядке убывания. В качестве вспомогательной структуры данных можно и нужно использовать второй дек. Сортировку производить, используя только стандартные операции работы с деком.

Схематичное изображение структуры данных:

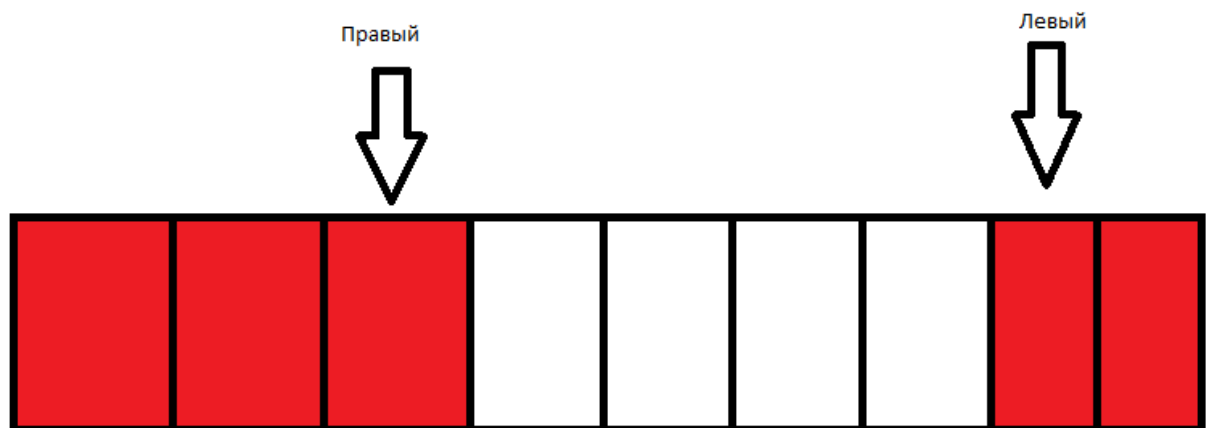


Схематичное изображение структуры хранения, использованной в программе:

Пустой дек:



Частично заполненный дек:



Заполненный дек:



Текст программы:

```
main.cpp

#include "mas.h"
#include <iostream>
using namespace std;

int main() {
    setlocale(LC_ALL, "ru");
    Mas* d; // создание дека
    int menu;
    d = new Mas; // выделение памяти под дек
    do {
        system("cls");
```

```

fflush(stdin);
cout << "1.Добавить элемент в левый конец\n2.Добавить элемент в
правый конец\n3.Удалить элемент с левого конца\n4.Удалить элемент с правого
конца\n5.Вывести крайний левый элемент\n6.Вывести крайний правый
элемент\n7.Дек полон?(O_o)\n8.Дек пуст?(o_O)\n9.Привести дек в
порядок\n10.Выйти и вывести содержимое дека" << endl;
cin >> menu;
switch (menu) {
case(1): {
    int n;
    do {
        system("cls");
        fflush(stdin);
        cout << "Введите натуральное число: " << endl;
        cin >> n;
    } while (n <= 0);
    system("cls");
    if (!d->AddBegin(n))
        cout << "Не удалось добавить элемент в дек!!!" << endl;
    else
        cout << "Элемент успешно добавлен в дек!!!" << endl;
    getchar();
    getchar();
}break;
case(2): {
    int n;
    do {
        system("cls");
        fflush(stdin);
        cout << "Введите натуральное число: " << endl;
        cin >> n;
    } while (n <= 0);
    system("cls");
    if (!d->AddEnd(n))
        cout << "Не удалось добавить элемент в дек!!!" << endl;
    else
        cout << "Элемент успешно добавлен в дек!!!" << endl;
    getchar();
    getchar();
}break;
case(3): {
    system("cls");
    if (!d->DelBegin())
        cout << "Не удалось удалить элемент в деке!!!" << endl;
    else
        cout << "Элемент успешно удалён!!!" << endl;
    getchar();
    getchar();
}break;
case(4): {
    system("cls");
    if (!d->DelEnd())
        cout << "Не удалось удалить элемент в деке!!!" << endl;
    else
        cout << "Элемент успешно удалён!!!" << endl;
    getchar();
    getchar();
}break;
case(5): {
    system("cls");
    int x = d->ReadBegin();
    if (x)
        cout << "Крайний левый элемент - " << x << endl;
}

```

```

        else
            cout << "Нет крайнего левого элемента!" << endl;
            getchar();
            getchar();
        }break;
    case(6): {
        system("cls");
        int x = d->ReadEnd();
        if (x)
            cout << "Крайний правый элемент - " << x << endl;
        else
            cout << "Нет крайнего правого элемента!" << endl;
        getchar();
        getchar();
    }break;
    case(7): {
        system("cls");
        if (d->Full())
            cout << "Дек полон!" << endl;
        else
            cout << "Дек не полон!" << endl;
        getchar();
        getchar();
    }break;
    case(8): {
        system("cls");
        if (d->Empty())
            cout << "Дек пуст!" << endl;
        else
            cout << "Дек не пуст!" << endl;
        getchar();
        getchar();
    }break;
    case(9): {
        system("cls");
        Mas *a = new Mas(d->maxlen);
        Sort(&a, &d);
        cout << "Сортировка выполнена!" << endl;
        getchar();
        getchar();
    }break;
    };
} while (menu != 10);
system("cls");
while (!d->Empty()) {
    cout << d->ReadBegin() << "\t";
    d->DelBegin();
}
delete d;
return 0;
}

```

```

mas.cpp
#include <iostream>
#include <cmath>
#include "mas.h"
Mas::Mas() {
    maxlen = 20; //Длина массива
    mas = new int[maxlen]; //выделяем память под длину массива
    left = 0; //голова
    right = maxlen - 1; // хвост
}

```

```

Mas::Mas(int n) {
    maxlen = n; // длина массива
    mas = new int[maxlen];
    left = 0;
    right = maxlen - 1;
}
bool Mas::Empty() { // проверка на пустоту
    return (right + 1) % maxlen == left;
}
bool Mas::Full() { // проверка на заполненность (+2 для того, чтобы можно
    было отличить пустой от заполненного )
    return (right + 2) % maxlen == left;
}
bool Mas::AddBegin(int x) { // добавление в начало
    if (Full()) // проверка на заполненность
        return false;
    left = left ? left - 1 : maxlen - 1; // если left !=0
    this->mas[left] = x; // в голову добавление
    return true;
}
bool Mas::AddEnd(int x) { // добавление в конец
    if (Full()) // проверка на заполненность
        return false;
    right = (right + 1) % maxlen;
    this->mas[right] = x; // добавление в хвост
    return true;
}
bool Mas::DelBegin() { // удаление первого элемента дека(головы)
    if (Empty()) // если не пустой
        return false;
    if (left == maxlen - 1) //удаление последнего добавленного в начало
        left = 0;
    else
        left += 1;
    return true;
}
bool Mas::DelEnd() { // удаление последнего элемента дека (хвост)
    if (Empty()) // если не пустой
        return false;
    if (right == 0) // удаление последеого добавленного в конец
        right = maxlen - 1;
    else
        right -= 1;
    return true;
}
int Mas::ReadBegin() { //считывание из головы
    if (Empty())
        return 0;
    else
        return mas[left];
}
int Mas::ReadEnd() { // считывание из хвоста
    if (Empty())
        return 0;
    else
        return mas[right];
}
Mas::~Mas() { // деструктор
    delete[] mas;
}
void Sort(Mas** a, Mas** b) { //передаем 2 дека
    while (! (* b)->Empty()) { //пока не пустой

```

```

        int x = (*b)->ReadBegin(); // в x записываем число из дека b(дек
b - изначальный дек)
        (*b)->DelBegin(); //переставляем left
        if (Check(x)) { //если число не простое
            if ((*a)->Empty()) // если пустой дек a записываем в голову
                (*a)->AddBegin(x);
            else {
                int i = 0;
                while (!(*a)->Empty() && x <= (*a)->ReadBegin()) {
//пока пустой и число <= послденего добавленного в дек a
                    (*b)->AddBegin((*a)->ReadBegin()); // в дек b
записываем последний добавленный в a тем самым освобождаем место в массиве b
для элемента меньше.
                    i += 1;
                    (*a)->DelBegin(); //переставляем left
                }
                (*a)->AddBegin(x); // добавление в начало дека a ,
если элемент x больше последнего добавленного в голову
                for (; i > 0; i--) { // элементы которые пришлось
перебросить в дек b, для освобождения места под элемент, возвращаем после
добавленного элемента
                    (*a)->AddBegin((*b)->ReadBegin());
                    (*b)->DelBegin();
                }
            }
        }
    }
    delete[] (*b)->mas; // удаляем дек b
    (*b)->left = (*a)->left; // переносим границы
    (*b)->right = (*a)->right;
    (*b)->mas = (*a)->mas; // переносим границы и отсортированные данные
}
bool Check(int x) { // проверка числа на простоту
    int q = (int)pow(x, 0.5);
    for (int i = 2; i <= q; i++)
        if (x % i == 0) // если остаток от деления равен 0, то число
простое
            return true;
    if (x == 1) // если остаток 1 то число не простое
        return true;
    return false;
}

```

mas.h

```

#pragma once
class Mas {
public:
    int left, right, maxlen;
    int* mas;
    Mas(); //конструктор
    Mas(int n);
    ~Mas(); //деструктор
    bool AddBegin(int x); //Добавление в начало
    bool AddEnd(int x); //Добавление в конец
    bool DelBegin(); //Удаление первого
    bool DelEnd(); //Удаление последнего
    int ReadBegin(); //Чтение элемента с начала
    int ReadEnd(); //Чтение элемента с конца
    bool Full(); //Проверка на заполнение
    bool Empty(); //Проверка на пустоту

```

```
};  
bool Check(int x); //Проверка на "простоту" числа  
void Sort(Mas** a, Mas** b); //сортировка дека
```

Результаты работы программы:

```
1.Добавить элемент в левый конец  
2.Добавить элемент в правый конец  
3.Удалить элемент с левого конца  
4.Удалить элемент с правого конца  
5.Вывести крайний левый элемент  
6.Вывести крайний правый элемент  
7.Дек полон?(O_o)  
8.Дек пуст?(o_O)  
9.Привести дек в порядок  
10.Выйти и вывести содержимое дека
```

Выбираем 8.

```
Дек пуст!
```

Выбираем 1.

```
Введите натуральное число:  
10_
```

Ввели число

```
Элемент успешно добавлен в дек!!!
```

Выводится сообщение об успешном добавлении

Выбираем 2.

```
Введите натуральное число:  
10_
```

Ввели число

```
Элемент успешно добавлен в дек!!!
```

Выбираем 5.

```
Крайний левый элемент - 12
```

Выбираем 6.

```
Крайний правый элемент - 10
```

Добавил еще несколько элементов 2 в начало и 2 в конец (в сумме 3 в начало и 3 в конец записано)

Выбираем 7.

```
Дек не полон!
```

Дек не полон т.к выделили размер дека 20 элементов

Выбираем 8.

Дек не пуст!

Выбираем 9.

Сортировка выполнена!

Выбираем 3.(удаления элемента с левого конца (из головы))

Элемент успешно удалён!!!

Выбираем 4.(удаления элемента с правого конца (из хвоста))

Элемент успешно удалён!!!

В итоге должны получить по 2 элемента с каждого из концов

Выбираем 10.

15 12 10 9

Идеально, отсортированный дек в порядке убывания, то о чем мечтали.